

1. Problema

Criar um programa em Python que implemente o jogo da forca. O jogador deve tentar adivinhar uma palavra secreta, recebendo feedback sobre acertos, erros e tentativas restantes.

2. Entrada

- Uma palavra secreta (pré-definida ou escolhida aleatoriamente).
- Letras digitadas pelo usuário.

3. Processamento

- Verificar se a letra está na palavra.
- Atualizar o estado do jogo (letras descobertas, erros, tentativas).
- Verificar vitória ou derrota.
- Exibir o progresso a cada jogada.

4. Saída Esperada

- Estado da palavra (ex: _ _ A _ _).
- Mensagens de acerto/erro.
- Número de tentativas restantes.
- Resultado final (ganhou/perdeu).

5. Prompt utilizado

PAPEL: Atue como desenvolvedor Python sênior. PROBLEMA: Criar a lógica de um jogo da forca simples. ENTRADA: Palavra secreta e letras digitadas pelo usuário. SAÍDA: Estado da palavra, tentativas restantes e resultado final. RESTRIÇÕES:

- Código legível.
- Sem bibliotecas externas.

- Separar verificação de letra em função própria. CRITÉRIOS DE QUALIDADE: clareza, modularização, tratamento básico de erros. CASOS DE TESTE: jogador acerta tudo, jogador erra tudo, jogador mistura acertos e erros.

6. Código inicial gerado pela IA

```
python

palavra = "python"

tentativas = 6

letras_descobertas = ["_"] * len(palavra)

while tentativas > 0:

    letra = input("Digite uma letra: ")

    if letra in palavra:

        for i in range(len(palavra)):

            if palavra[i] == letra:

                letras_descobertas[i] = letra

        print("Acertou!")

    else:

        tentativas -= 1

        print("Errou! Tentativas restantes:", tentativas)

        print("Estado atual:", " ".join(letras_descobertas))

        if "_" not in letras_descobertas:

            print("Você venceu!")

            break

        if "_" in letras_descobertas:

            print("Você perdeu! A palavra era:", palavra)
```

7. Análise crítica da primeira versão

Pontos positivos

- Funciona.
- Lógica clara.

- Estrutura simples.

Problemas encontrados

- Não há função separada para verificar letras (falta modularização).
- Não trata entradas inválidas (ex: números, strings longas).
- Palavra fixa — não há variedade.
- Código repetitivo no laço.
- Não há documentação.

8. Casos de Teste

Teste 1 — Caso normal

Entrada: p, y, t, h, o, n Resultado: vitória Funcionamento: correto

Teste 2 — Caso limite

Entrada: letras erradas até acabar tentativas Resultado: derrota Funcionamento: correto

Teste 3 — Entrada inválida

Entrada: "ab", "3", "" Resultado: programa aceita, mas não deveria Funcionamento: precisa melhorar

9. Problemas encontrados

- Falta modularização.
- Falta validação de entrada.
- Palavra fixa.
- Código sem docstring.
- Repetição de lógica dentro do loop.

10. Prompt de refatoração

Prompt: "Refatore o código abaixo. O programa já funciona. Melhore clareza, modularização, nomes de variáveis e tratamento de erros. Não altere o comportamento esperado. Explique cada alteração."

11. Código refatorado pela IA (versão final)

```
python

import random

def escolher_palavra():

    palavras = ["python", "computador", "programacao", "desenvolvimento"]

    return random.choice(palavras)

def validar_letra(letra):

    return len(letra) == 1 and letra.isalpha()

def atualizar_estado(palavra, letra, estado):

    for i, char in enumerate(palavra):

        if char == letra:

            estado[i] = letra

def jogo_da_forca():

    palavra = escolher_palavra()

    tentativas = 6

    estado = ["_"] * len(palavra)

    letras_usadas = set()

    print("Bem-vindo ao Jogo da Forca!")

    while tentativas > 0:

        print("\nPalavra:", " ".join(estado))

        print("Tentativas restantes:", tentativas)

        print("Letras usadas:", ", ".join(sorted(letras_usadas)))

        letra = input("Digite uma letra: ").lower()

        if not validar_letra(letra):
```

```

    print("Entrada inválida. Digite apenas uma letra.")

    continue

if letra in letras_usadas:

    print("Você já tentou essa letra.")

    continue

letras_usadas.add(letra)

if letra in palavra:

    atualizar_estado(palavra, letra, estado)

    print("Boa! Você acertou uma letra.")

else:

    tentativas -= 1

    print("Letra incorreta.")

if "_" not in estado:

    print("\nParabéns! Você venceu!")

    print("A palavra era:", palavra)

    return

print("\nVocê perdeu!")

print("A palavra era:", palavra)

jogo_da_forca()

```

12. Comparação entre versões

Critério	Inicial Refatorada	
Funcionamento	5	5
Clareza	3	5
Organização	2	5
Legibilidade	3	5

Critério	Inicial Refatorada	
Tratamento de erros	1	4
Facilidade de manutenção	2	5

13. Reflexão

- Onde a IA mais ajudou? Na refatoração e modularização do código.
- Onde a IA errou? Na primeira versão, não tratou entradas inválidas.
- O que precisei modificar? Separar funções, validar entrada, melhorar nomes.
- Consigo explicar o código? Sim — cada função tem responsabilidade clara.

14. Take Away

Programar com IA não significa deixar a IA programar por mim. Significa compreender, revisar, testar e assumir responsabilidade por cada linha gerada.